

ControlPlane Checker

Build Plan & Technical Specification

Team DLS · Accenture Innovation Challenge 2026

Prepared for: Narain Gopinath (Lead), Raghav Ram (Frontend), Joshua John (Backend)

Document date: 27 Aug 2026 — **3 days to deadline: 30 Aug 2026, 11:59 PM IST**

This document is a detailed, buildable plan for ControlPlane Checker: a model-agnostic proxy that scores every LLM response live on performance, cost, and responsibility, routes it by severity, and correlates signals across checks to surface compound failures. It expands the project handoff into concrete tasks, file-level responsibilities, data contracts, and a day-by-day schedule against the 30 Aug deadline — now compressed to the 3 days remaining as of 27 Aug.

1. Purpose, Goals & Scope

1.1 Problem statement

AI deployments fail silently: responses can be wrong, expensive, or unsafe, and teams only find out after damage is done (bad decisions made, budget burned, policy violated). There is no live control plane that scores a response as it happens and intervenes before it reaches the end user.

1.2 What we are building

A model-agnostic proxy layer. Every LLM response passed through it is scored live across three independent check dimensions, then a severity router decides its fate (pass / log / block-for-review). A correlation engine looks across the three check outputs on the same response to catch compound failures — e.g. a cost spike landing on the same response as a low-confidence score — that none of the three checks would flag alone. Results stream live to a dashboard.

1.3 Success criteria

- End-to-end pipeline demoable live: request → score → route → dashboard update.
- Cross-signal correlation actually implemented, not just three isolated checks running in parallel.
- README lets a stranger clone and run the whole stack in under 5 minutes.
- Repo public, demo video recorded, submitted before 30 Aug 2026, 11:59 PM IST.

1.4 Explicitly out of scope this sprint

- Auto-remediation or auto-editing of flagged responses — document as future work only.
- Multi-tenant auth — single-tenant/local demo only.
- Production-grade horizontal scaling — SQLite + single Docker Compose stack is sufficient.

2. Technical Architecture

2.1 Stack

Layer	Choice	Notes
Backend	FastAPI (Python 3.11)	async-native, WebSocket support built in
Frontend	React + Vite + TypeScript + Tailwind v3	fast dev loop, typed
Database	SQLite	Postgres swap documented in README, not built
Orchestration	Docker Compose	single local environment, no cloud hosting
CI	GitHub Actions	lint on every PR; tests added once backend/tests/ has content

2.2 Request lifecycle

A client (or the demo seed script) sends a prompt/response pair to the proxy. The proxy fans it out to three checks concurrently, collects their scores, runs correlation across the three results, computes a final severity, persists the record, and pushes it to all connected dashboard clients over WebSocket. The severity router's decision (pass/log/block) is returned synchronously to the caller and also broadcast live.

```

client
-> POST /score {prompt, response, meta}
-> proxy.py: run checks.performance + checks.cost + checks.responsibility (async)
-> correlation.py: inspect the 3 results together, add compound flags
-> router.py: map (checks + correlation) -> severity {pass|log|block}
-> db.py: persist ScoredResponse row
-> ws.py: broadcast the scored record to all connected dashboards
<- 200 {severity, checks, correlation, decision}

```

2.3 File structure

```

controlplane-checker/
backend/app/
main.py, proxy.py, router.py, correlation.py
checks/ (performance.py, cost.py, responsibility.py)
ws.py, db.py, models.py
frontend/src/ (App.tsx, components/, hooks/)
demo/seed_transactions.py
docker-compose.yml
.github/workflows/ci.yml

```

2.4 Data contract

Both REST responses and WebSocket broadcast messages use this exact shape, so the dashboard renders identically whether it receives history via REST or a live push. Agree this before building the frontend and backend halves in isolation.

```

{
  "id": "uuid",
  "timestamp": "ISO-8601",
  "prompt": "string (truncated)",
  "response": "string (truncated, redacted if PII found)",
  "checks": {
    "performance": {"score": 0-100, "latency_ms": int, "flags": [str]},
    "cost": {"score": 0-100, "est_tokens": int, "est_cost_usd": float, "flags": [str]},
    "responsibility": {"score": 0-100, "pii_found": bool, "toxicity": 0-1, "flags": [str]}
  },
  "correlation": {"compound_flags": [str], "correlation_score": 0-100},
  "severity": "pass | log | block",
  "decision_reason": "string"
}

```

3. Check Design

3.1 Deterministic checks (cost, responsibility)

These do not require a model call, so they are fast to build and fast to demo reliably.

cost.py

- Estimate tokens from prompt+response length (heuristic tokenizer: char-count/4 fallback).
- Compute est_cost_usd from a configurable per-model \$/1K-token table (config dict, not hardcoded).
- Flag: cost spike if est_cost_usd exceeds a rolling-average threshold for that model.
- Score: 100 minus normalized overage; below-threshold responses score high.

responsibility.py

- PII detection: regex set for email, phone, card-like numbers, national ID patterns — redact before storing, never log raw PII (handoff Section 7 requirement).
- Toxicity/policy heuristic: keyword/category list (deterministic, no model call) scored 0-1.
- Restricted-category screen: mirrors ethical screens used on the finance side (defense, weapons, tobacco, alcohol, gambling), kept as an extensible flag list rather than hardcoded to finance.
- Score: composite of PII-clean + low-toxicity + no policy hits.

3.2 Performance check (async/heuristic)

- Latency: measured directly from request timing.
- Confidence heuristic: response length vs prompt complexity, refusal/hedge-phrase detection, repetition ratio — deterministic heuristics, not a second LLM call, to stay fast and demo-reliable.
- Score: weighted combination of latency-in-budget + heuristic confidence.

3.3 Severity router

Condition	Severity	Action
All three scores healthy, no correlation flags	pass	log only, no intervention
One check below threshold, or minor correlation flag	log	pass through, flagged for review
Any check critically low (PII found, cost far over budget), or compound flag	block	response withheld, routed to human review

Thresholds live in a single config module so they are tunable without touching check logic — important for demo tuning in the final rehearsal.

3.4 Correlation engine (the differentiator)

This is the highest-risk, highest-value piece per the handoff's own risk note. Build it as a small rule set operating on the three check outputs for a single response, not a separate model:

- Rule: cost spike + low confidence → likely the model is rambling/retrying to justify cost; `compound_flag = "cost_confidence_mismatch"`.
- Rule: high latency + PII found → slow response paths correlate with unsanitized output in testing; `compound_flag = "latency_pii_correlation"`.
- Rule: repeated compound flags on the same session within a time window → escalate severity even if any single response would only warrant 'log'.
- Fallback if time is short: ship 2 solid rules instead of a generic correlation framework — a working differentiator beats an unfinished general engine.

4. Dashboard & Frontend

4.1 Components

Component	Responsibility
App.tsx	Layout, WebSocket connection lifecycle, global state
hooks/useLiveFeed.ts	Opens WS to /ws, appends incoming records, exposes connection status
components/Dashboard.tsx	List/grid of scored responses, most recent first

Component	Responsibility
components/ResponseCard.tsx	One scored response: excerpt, 3 check scores, correlation flags
components/SeverityBadge.tsx	Color-coded pass/log/block pill

4.2 Live feed behavior

- On mount: fetch recent history via REST (GET /history) to populate initial state.
- Then: open WebSocket to /ws and prepend new records as they arrive.
- Reconnect with backoff if the socket drops — must survive a demo hiccup live on stage.

5. Milestones & Timeline — 3 Days Remaining

Status as of 27 Aug (today): scaffold, all three checks, severity router, correlation engine, FastAPI backend (REST + WebSocket), React dashboard, Docker Compose, and CI are already built and passing tests. The schedule below reflects what is actually left, not the original 14-day estimate from the handoff.

Date	Day	Focus	Owner
27 Aug (today)	Day 1	Integration pass: run the full stack end-to-end via Docker Compose, fire demo/seed_transactions.py against it, fix anything that breaks on a clean clone. Finalize the WebSocket payload schema if not already locked.	Joshua + Raghav
28 Aug	Day 2	Write and rehearse the demo script; record the demo video showing request -> score -> route -> live dashboard update, including at least one correlation compound-flag firing on screen.	Narain
29 Aug	Day 3	Buffer day: fix anything the rehearsal surfaced, polish README, confirm repo is public with no committed secrets, re-run CI on a fresh PR.	All
30 Aug	Deadline	Final checks and submit before 11:59 PM IST. No new features after midday — only fixes to what's already built.	Narain (submission owner)

Risk carried over from the original handoff: the correlation engine was the most technically demanding piece and the stated differentiator. It is now built and unit-tested (Section 3.4), which removes the single biggest risk to the 3-day runway — remaining time should go to integration, rehearsal, and the video, not new features.

6. Development Standards

Area	Standard
Branching	main (protected, demo-stable), fed by dev (integration), fed by feat/* branches. No direct pushes to main.
PR process	One reviewer approval before merging to dev; merge dev into main only at milestone checkpoints.
Lint/format	ruff + black (Python), eslint + prettier (JS).
Tests	backend/tests/test_checks.py covers checks, correlation, and router; run locally with pytest before every PR.

7. CI/CD, Data & Security

- GitHub Actions runs on every PR: install backend deps, run ruff check (and pytest, now that backend/tests/ has content). No deploy step (local-only project).
- Single local Docker Compose environment; no staging/prod split needed for this scope.
- No rollback mechanism needed — no live deployment. If main breaks, revert the merge commit.
- Repo is public (challenge requirement) — no secrets ever committed; .env.example documents required variables without real values.
- Demo mode uses mocked/seeded LLM responses — no real user data or API keys required to run the core demo.
- If wiring a real LLM call: use your own key locally via .env (gitignored), never commit it.
- The tool's own PII-detection feature redacts before storing — raw PII is never logged into its own demo database.

8. Onboarding & Demo Script

```
git clone https://github.com/JoshuaCoutinho-AI/controlplane-checker.git
cd controlplane-checker
git checkout dev
docker compose up --build
```

- Backend: `http://localhost:8000/health -> {"status":"ok"}`
- Frontend: `http://localhost:5173 -> renders dashboard`
- No .env required for demo mode; copy .env.example to .env only if wiring a real LLM key
- Demo flow: run `demo/seed_transactions.py` to fire a burst of mixed good/bad/borderline responses through the proxy while the dashboard is open, so the live feed, severity routing, and at least one correlation compound-flag are all visible on screen within about 60 seconds.

9. Immediate Next Actions (in order)

- Run `docker compose up --build` from a clean clone today and confirm the 5-minute README promise actually holds — this is the single highest-value check with 3 days left.
- Run `demo/seed_transactions.py` against it and confirm at least one response hits each of pass, log, and block, and that a `compound_flag` fires at least once.
- Lock the demo script and record the video on Day 2 — do not leave video recording for Day 3.
- Freeze new feature work after Day 2; Day 3 is fixes and submission only.